

Automata

languages lexers, parsers, interpreters

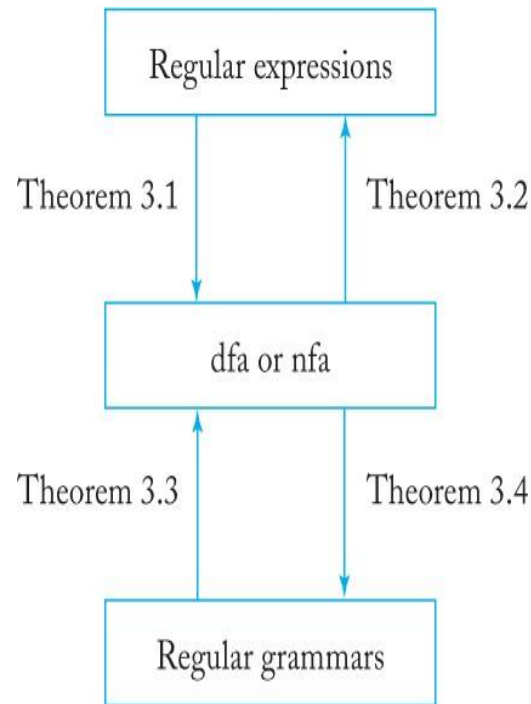


Context-free languages

Agenda

- Recap
- Context-free grammars
- Parsing and ambiguity
- Context-free grammars and programming languages

Recap: Regular languages and regular grammars



Context-free grammars

A grammar $G = (V, T, S, P)$ is said to be

- **right-linear** if all productions are of the form
 - $A \rightarrow xB$,
 - $A \rightarrow x$, ($A, B \in V$, and $x \in T^*$)
- **context-free** if all productions in P have the form
 - $A \rightarrow y$ ($A \in V$ and $y \in (V \cup T)^*$)

Context-free grammars

- Substitution variable on the left of a production does not depend on the sentential form (the context)
- Every regular grammar is context-free
- Context-sensitive grammar rules are of the form
 - $\alpha A \beta \rightarrow \alpha \gamma \beta$

Context-free sample

Production rules

- $S \rightarrow aSa,$
- $S \rightarrow bSb,$
- $S \rightarrow \lambda$

Typical derivation

- $S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabSbaa \Rightarrow aabbbaa$

Generated language

- $L(G) = \{ww^R : w \in \{a, b\}^*\}.$

Excercise

Let $L = \{ a^n b^n : n \text{ is odd} \}$

Question: is language L context-free?

Find a context-free grammar which produces L .

Leftmost and Rightmost Derivations

Consider grammar with productions

- $S \rightarrow aAB,$
- $A \rightarrow bBb,$
- $B \rightarrow A|\lambda.$

Give derivation for “abbbb”

$S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abbbbB \Rightarrow abbbb$ (left)

$S \Rightarrow aAB \Rightarrow aA \Rightarrow abBb \Rightarrow abAb \Rightarrow abbBbb \Rightarrow abbbb$ (right)

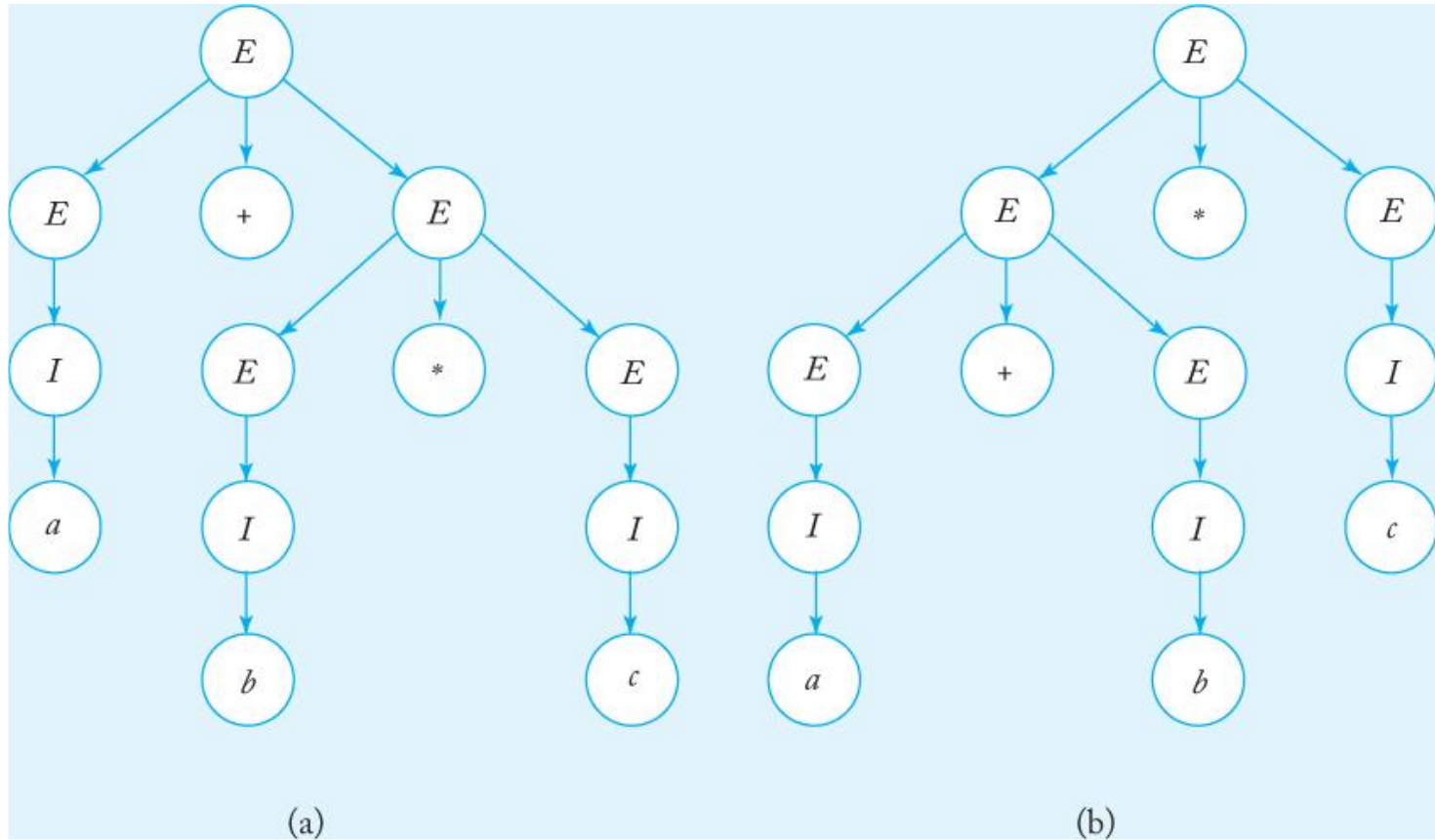
Both derivations have the same derivation tree

Ambiguity

Consider grammar with productions

- $E \rightarrow I,$
- $E \rightarrow E + E,$
- $E \rightarrow E^*E,$
- $E \rightarrow (E),$
- $I \rightarrow a|b|c.$

Two derivation trees for $a+b*c$



Solving ambiguity

- Associate precedence rules with operators
 - Is outside the grammar
- Rewrite grammar

$$E \rightarrow T,$$
$$T \rightarrow F,$$
$$F \rightarrow I,$$
$$E \rightarrow E + T,$$
$$T \rightarrow T * F,$$
$$F \rightarrow (E),$$
$$I \rightarrow a|b|c.$$

- Antlr?

Backus-Naur Form (BNF)

$\langle \text{expression} \rangle ::= \langle \text{term} \rangle | \langle \text{expression} \rangle + \langle \text{term} \rangle$

$\langle \text{term} \rangle ::= \langle \text{factor} \rangle | \langle \text{term} \rangle * \langle \text{factor} \rangle$

Parsing and membership

Consider grammar with productions

$$S \rightarrow aAB; A \rightarrow bBb; B \rightarrow A|\lambda$$

Question: does a given string w of terminals belong to $L(G)$?

- Approach brute force parsing
 - Problems:
 - Efficiency
 - Never terminates

Efficiency

- For every context-free grammar there exists an algorithm that parses any $w \in L(G)$ in a number of steps proportional to $|w|^3$.
- Not acceptable for a moderately long program.

Simple grammar

A context-free grammar $G = (V, T, S, P)$ is said to be a simple grammar or s-grammar if all its productions are of the form

$$A \rightarrow ax,$$

where $A \in V$, $a \in T$, $x \in V^*$, and any pair (A, a) occurs at most once in P .

- A s-grammar can be parsed in linear time.
- Many features of common programming can be described in s-grammars